

Prompt Documentation

Prompt Documentation

Purpose Of This Document

The coursework brief asks for a document explaining what prompts were used and for what specific knowledge-engineering task. The repository preserves one operational LLM prompt verbatim and preserves evidence of a few additional prompt-driven activities indirectly through comments, outputs, and cached files. This document separates:

- prompts recoverable exactly from the repository
- prompt-driven tasks that are only reconstructable from repository evidence

Where a prompt is reconstructed rather than preserved verbatim, that is stated explicitly.

Exact Prompt Recoverable From The Repository

Task: triple extraction from unstructured text

Source: `data_retrieval/unstructured_retrieval/extract_triples.py`

Model/runtime recorded in code:

- local Ollama endpoint at `http://localhost:11434/api/generate`
- model name: `mistral`
- caching enabled through `data/caches/triples_cache.json`
- current repository setting: `USE_LLM = False`

Repository-preserved prompt:

Instruction

Extract RDF triples from the text below about London public transport.

A triple has the format: `subject | predicate | object`

Constraints

- ONLY extract facts explicitly stated in the text
- Do NOT infer, guess, or add external knowledge
- Use clean short labels: "Victoria line" not "the Victoria line"
- Use camelCase predicates: "operatedBy" not "operated by"
- Use natural labels with spaces: "Elizabeth line" not "ElizabethLine" or "Elizabeth_line"
- Do NOT prefix with "subject:" or "predicate:" or "object:"
- Maximum 10 triples
- If no facts can be extracted, return: NONE

Examples

Victoria line | operatedBy | Transport for London

DLR | openedIn | 1987

```
Jubilee line | hasStop | Westminster  
Oyster card | acceptedOn | London Underground  
Zone 1 | contains | Kings Cross St Pancras
```

```
### Text  
{passage}
```

```
### Triples
```

Knowledge-engineering task supported:

- extraction of candidate subject-predicate-object statements from textual sources
- preparation of unstructured facts for later RDF mapping

Why this prompt is useful:

- it constrains the model to explicit facts
- it normalizes predicate style for later mapping
- it limits output length
- it supports downstream validation against spaCy-recognized entities

Prompt-Driven Activities Evidenced Indirectly

Task: predicate-to-ontology mapping refinement

Source evidence:

- `data_retrieval/unstructured_retrieval/triples_to_rdf.py`
- comment: "using output of this script ... and an LLM i made this map"

What is preserved:

- the final predicate map

What is not preserved:

- the exact prompt used to create or refine the map

Repository-consistent reconstructed prompt:

Given the following list of extracted free-text predicates from London transport articles, r

Knowledge-engineering task supported:

- ontology alignment between noisy LLM extractions and the project's formal property vocabulary

Status:

- reconstructed from comments and outputs, not preserved verbatim

Task: generation of additional competency questions

Source evidence:

- coursework brief requires 10 manually created and 10 LLM-augmented competency questions
- `README.md` preserves a manual/LLM split
- `competency_questions/competency_question.txt` preserves the final 20 SPARQL-backed questions

What is preserved:

- final question outputs

What is not preserved:

- the exact prompt used to generate the LLM-augmented question set

Repository-consistent reconstructed prompt:

Generate 10 additional competency questions for a knowledge graph about London public transport

Knowledge-engineering task supported:

- requirements expansion
- broadening the CQ set beyond straightforward operational questions

Status:

- reconstructed from the final question outputs, not preserved verbatim

Task: source discovery

Source evidence:

- coursework brief explicitly allows LLMs and web search to discover sources
- `docs/generated/tfl-sources.json` records the final sources that were selected

What is preserved:

- selected source URLs and categories

What is not preserved:

- the exact search prompts or web queries used to find those sources

Repository-consistent reconstructed prompt:

Find authoritative structured and textual data sources for a knowledge graph about London public transport

Knowledge-engineering task supported:

- source selection
- balancing authoritative operational data with textual explanatory sources

Status:

- reconstructed from the selected source manifest, not preserved verbatim

Task: ontology discovery

Source evidence:

- the coursework brief requires two existing ontologies
- the final ontology imports GTFS and Schema.org

What is preserved:

- the chosen ontologies and the resulting subclass/subproperty alignments

What is not preserved:

- the exact discovery prompts or search strings used to find GTFS and Schema.org

Repository-consistent reconstructed prompt:

Identify two existing ontologies that can be reused for a knowledge graph about London public transport.

Knowledge-engineering task supported:

- ontology reuse
- design justification for extending rather than inventing a transport vocabulary from scratch

Status:

- reconstructed from the ontology imports and alignments, not preserved verbatim

Prompt Execution Notes

The repository currently uses cached LLM outputs rather than live generation during a normal run:

- `USE_LLM = False` in `extract_triples.py`
- `data/caches/triples_cache.json` contains 563 cached extraction entries

This is good for reproducibility in a marking environment, but it also means the exact dialogue history that originally produced the cache is not fully recoverable from the repository alone.

Conclusion

The repository preserves one exact operational prompt and preserves the outputs of several other prompt-dependent tasks, but it does not preserve every search query or chat transcript used during knowledge engineering. For submission purposes, the safest and most honest position is:

- present the triple-extraction prompt verbatim
- identify the other prompt-driven tasks clearly
- label reconstructed prompts as reconstructed rather than original logs

That approach keeps the documentation complete without inventing provenance that the repository does not actually contain.